

Devotopia

Verified Skill Roadmaps

A guided walkthrough of the product and the code behind it.

Every screenshot in this document was captured from the application running locally,
and every figure is traced to a source file.

Graduation Project — Open Source Track — Alexandria

1. What Devotopia is

Devotopia turns a target job role into an ordered learning roadmap, then makes the learner prove each step. Every module is gated by an adaptive exam: pass it and the modules that depend on it unlock and a verified skill is written to the learner's passport; fail it and a shorter remedial module is generated from the exact questions that were missed. Companies read the same data in reverse, browsing candidates ranked by exam performance rather than by CV keywords.

The difference from a course catalogue is where the proof comes from. A completion certificate says a learner attended. A Devotopia verified skill says they answered questions at a measured difficulty and passed a weighted threshold.

The project in numbers

Repository layout	Turborepo monorepo, npm workspaces — apps/api, apps/web, packages/shared
Backend	NestJS 11 · 29 feature modules · 118 mapped HTTP routes · ~14,400 lines
Frontend	Next.js 14 App Router · 29 page routes · ~22,800 lines
Database	MongoDB via Mongoose · 35 schemas
Vector store	Qdrant, cosine similarity, collections verified on boot
AI layer	5 providers behind one fallback chain, ending in a deterministic mock
Languages	English and Arabic, with RTL layout switching and light/dark themes

How the pieces fit together

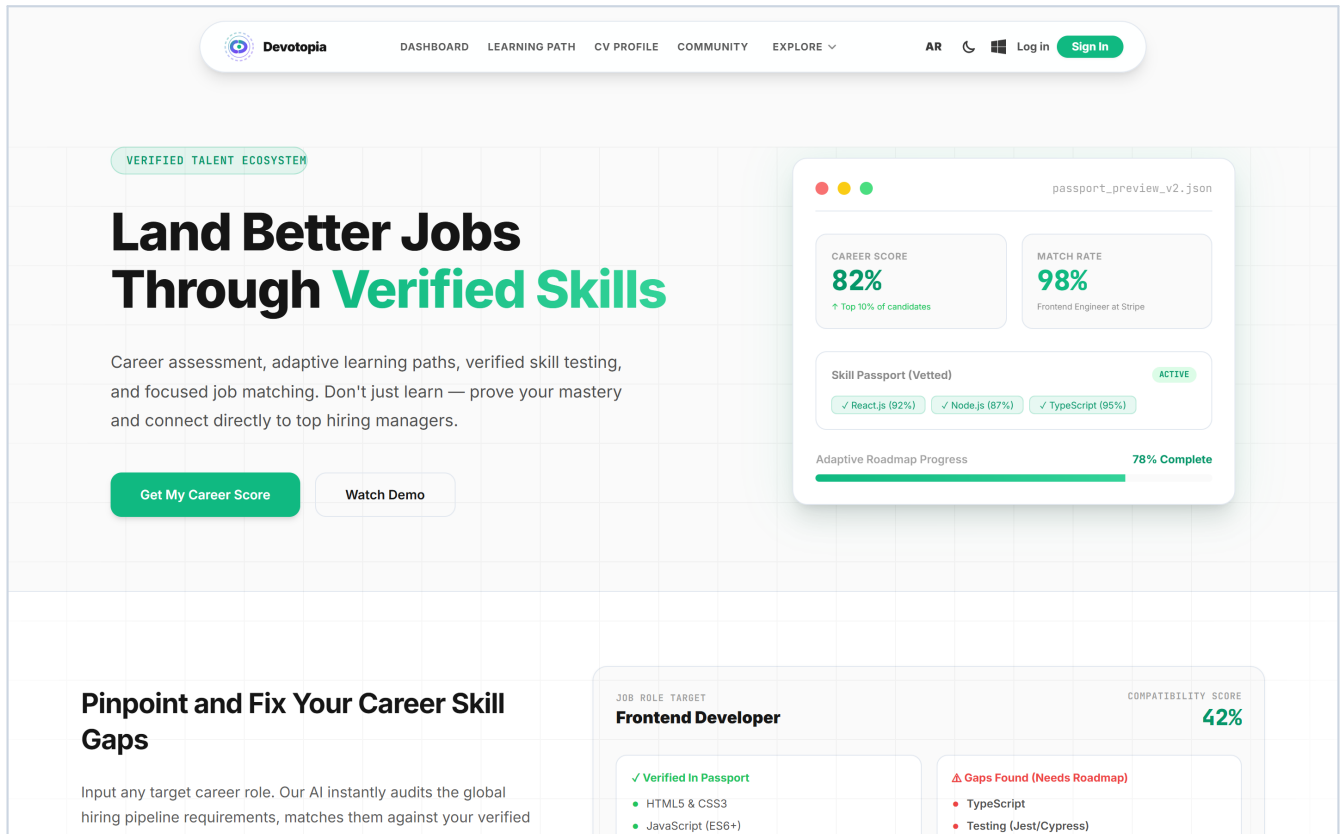
```
Browser (Next.js 14, App Router)
|
|  apiFetch() — access token in memory, refresh token in an httpOnly cookie
v
NestJS 11  ■■■  JwtAuthGuard (global, deny by default)
|
|  +■■■  RolesGuard          learner / company / admin / mentor
|
|  +■■■  Controller ■■■  Service ■■■■■■ MongoDB (Mongoose, 35 schemas)
|
|                                     ■■■■  LLMService ■■■  provider chain
|                                     |
|                                     |  OpenAI → Gemini → Groq
|                                     |  → HuggingFace → Mock
|
|                                     ■■■■  RAGService ■■■  Qdrant (cosine)
```

The request path. The guard runs before every controller unless a route opts out.

2. First contact — the public site

LANDING PAGE

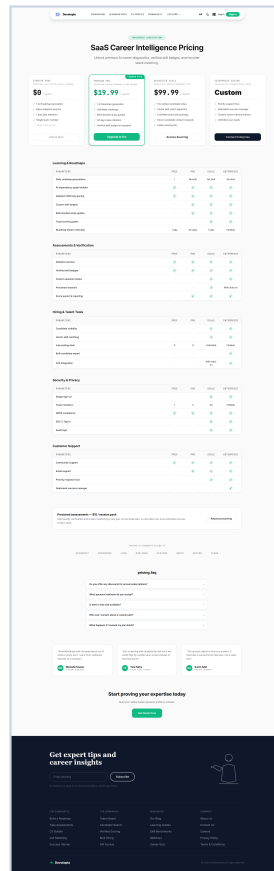
The entry point states the promise and routes visitors to registration. The navigation bar carries the language toggle (AR/EN), the theme switch, and the install prompt for the progressive web app — all three are present on every screen that follows.



The landing page at 1440×900. Language, theme and install controls sit in the navbar.

PRICING

Three tiers. The important detail is not on this page but behind it: the client never sends a price. It sends a plan name, and the server resolves the amount from a table before it creates the PayPal order, so a tampered request cannot buy a plan cheaply.



Pricing page. Plan amounts are resolved server-side in `payment.dto.ts`.

```
// apps/api/src/modules/payment/dto/payment.dto.ts
export const PLAN_PRICES: Record<'pro_learner' | 'company_tier', number> = {
  pro_learner: 19.99,
  company_tier: 99.99,
};

export class CreateOrderDto {
  // The price is resolved SERVER-side from this plan – the client never sends an amount.
  @IsIn(['pro_learner', 'company_tier'])
  plan!: 'pro_learner' | 'company_tier';
}
```

The comment in the source states the rule; the DTO enforces it by accepting only a plan name.

AUTHENTICATION

Registration and login share a token model built to survive an XSS incident. The refresh token is an httpOnly cookie scoped to /auth, so page JavaScript can never read it. The access token lives in a module variable and dies with the tab; on reload the app silently mints a new one from the cookie.

→ New here? [Sign up](#)

Sign In

Access your personalized tech syllabus path.

Email Address

Enter your email

Password

Remember me [Forgot Password?](#)

[Sign In](#)

[Facebook](#) [Google](#)

Don't have an account? [Sign up](#)

ENG [FAQ](#) [Support](#)

Login. The same screen handles Google identity sign-in.

→ Already have an account? [Sign in](#)

STEP 1 OF 2 • CREDENTIALS CONFIGURATION

Register

Create your account to start mapping your curriculum.

I want to register as

[Learner](#) [Recruiter](#)

Full Name

Daniel Ahmadi

Email Address

Enter your email

Password

[Continue to Onboarding](#)

Already have an account? [Sign in](#)

ENG [FAQ](#) [Support](#)

Registration. Role is selectable, but 'admin' is rejected by the DTO.

```
// apps/api/src/modules/auth/dto/auth.dto.ts
// NOTE: 'admin' is intentionally NOT assignable through public registration.
@IsOptional()
@IsIn(['learner', 'company', 'mentor'])
role?: 'learner' | 'company' | 'mentor';
```

Privilege escalation is closed at the validation layer, not in business logic.

3. The learner journey

STEP 1 — ONBOARDING

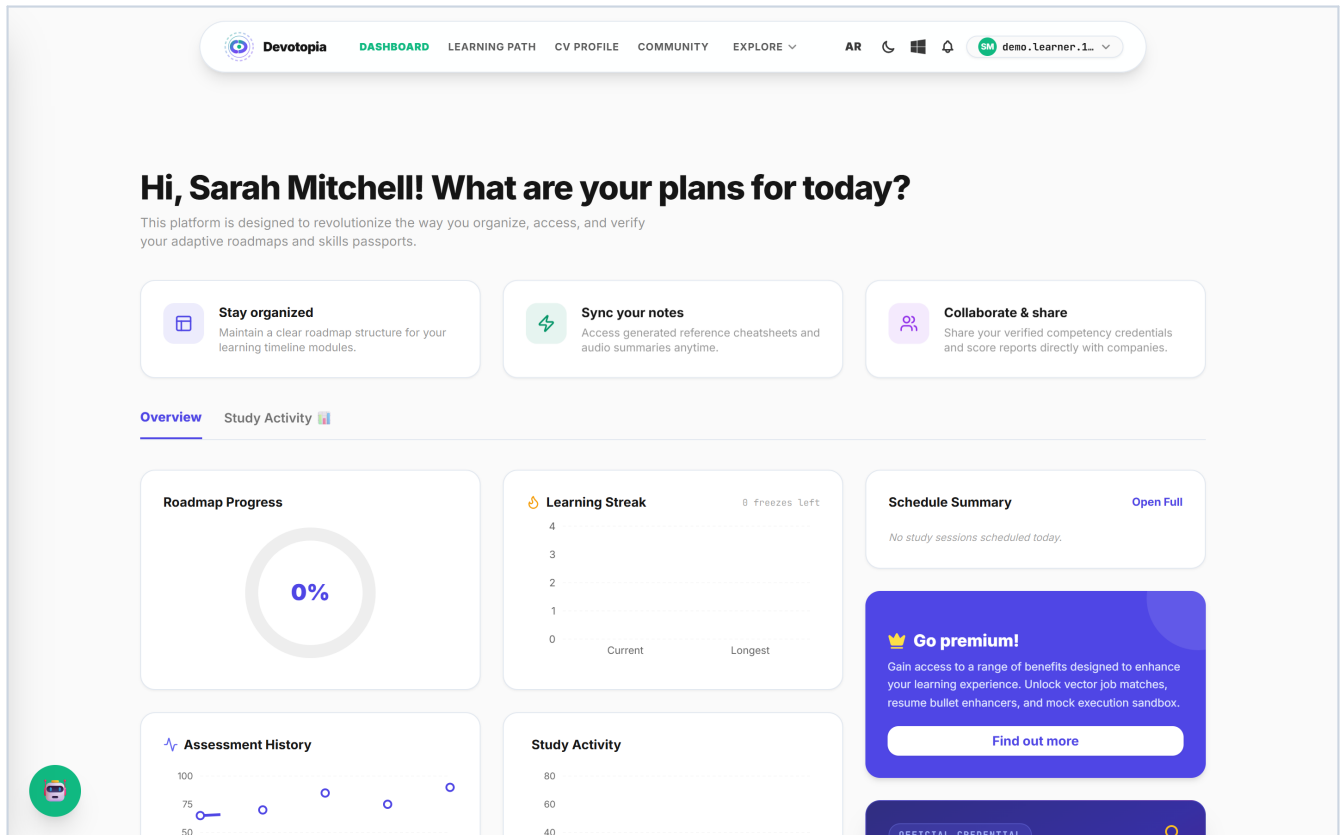
A short wizard collects the four inputs the curriculum generator needs: target role, education, years of experience, and current skills. Nothing else is asked, because nothing else changes the output.

The screenshot shows the Devotopia onboarding wizard. At the top is a navigation bar with the Devotopia logo, links for DASHBOARD, LEARNING PATH, CV PROFILE, COMMUNITY, and EXPLORE, along with user controls for AR, a Windows icon, a bell icon, and a user profile dropdown labeled 'demo.Learner.1...'. Below the navigation bar is a progress indicator with four steps: 1. Goal Selection (highlighted with a green circle), 2. Background, 3. Skills Mapping, and 4. Generating. The main content area is titled 'Personalize Your Syllabus Path' and includes the text 'SmartRoadmap generates a tailored technical path to prepare you directly for this role.' There are four role selection cards: 'Frontend Engineer' (Build reactive user interfaces, state managers, and modern layouts), 'Backend Developer' (Design microservices, API architectures, databases, and message queues), 'Data Scientist' (Process datasets, design machine learning algorithms, and build models), and 'DevOps Engineer' (Deploy cloud infrastructure, orchestrate containers, and automate CI/CD). Below these cards is a section 'OR DEFINE A CUSTOM ROLE' with a text input field containing 'e.g. Cloud Security Engineer'. A green 'Continue →' button is at the bottom right. A small circular icon with a robot face is in the bottom left corner.

Onboarding. These four answers become the prompt for roadmap generation.

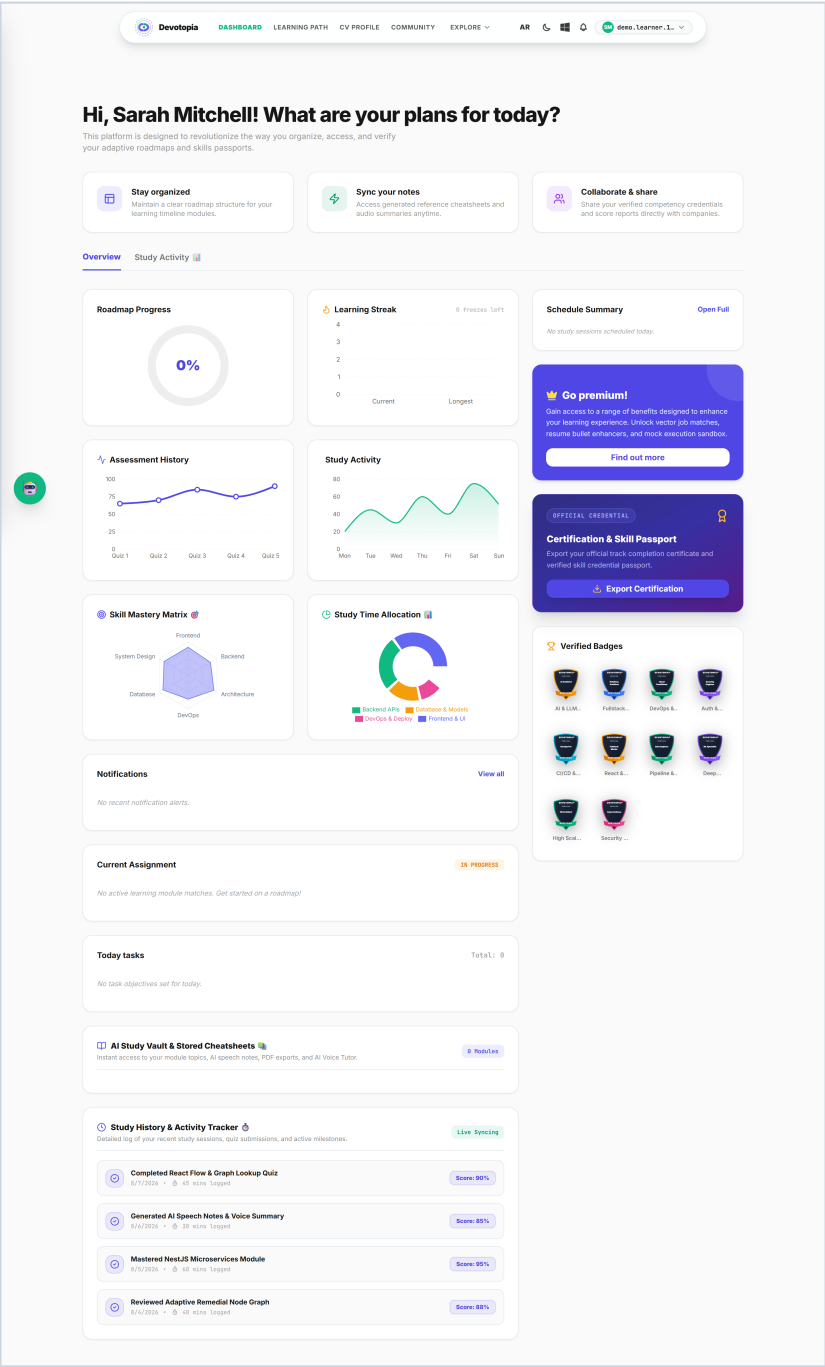
STEP 2 — THE DASHBOARD

After generation the learner lands here. Roadmap progress, the learning streak, assessment history and study activity are all live queries, not decoration — the streak and the achievement events are written by the same code path that finishes an exam.



Learner dashboard. Charts are Recharts, fed from the progress and streak modules.

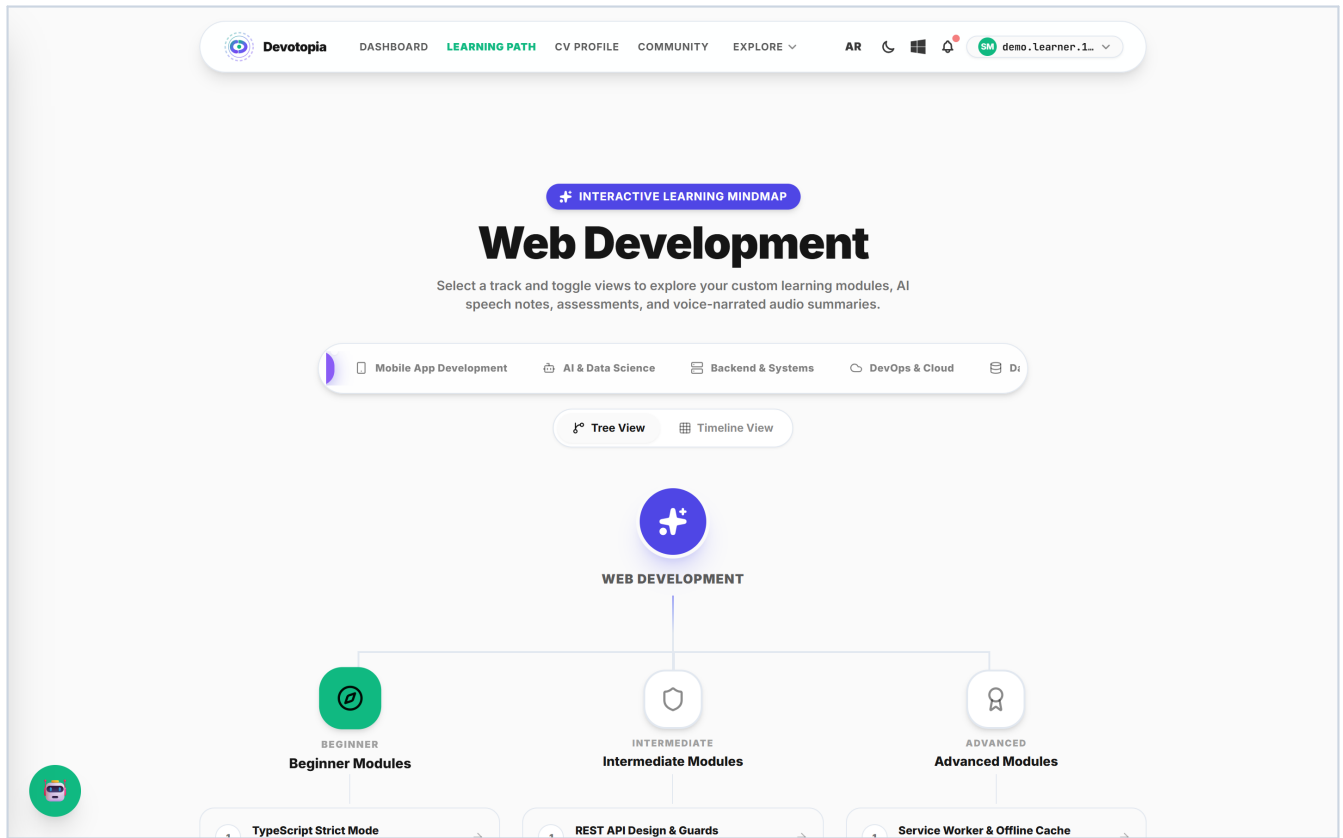
Scrolled out, the dashboard also carries the credential card and the recommended next actions:



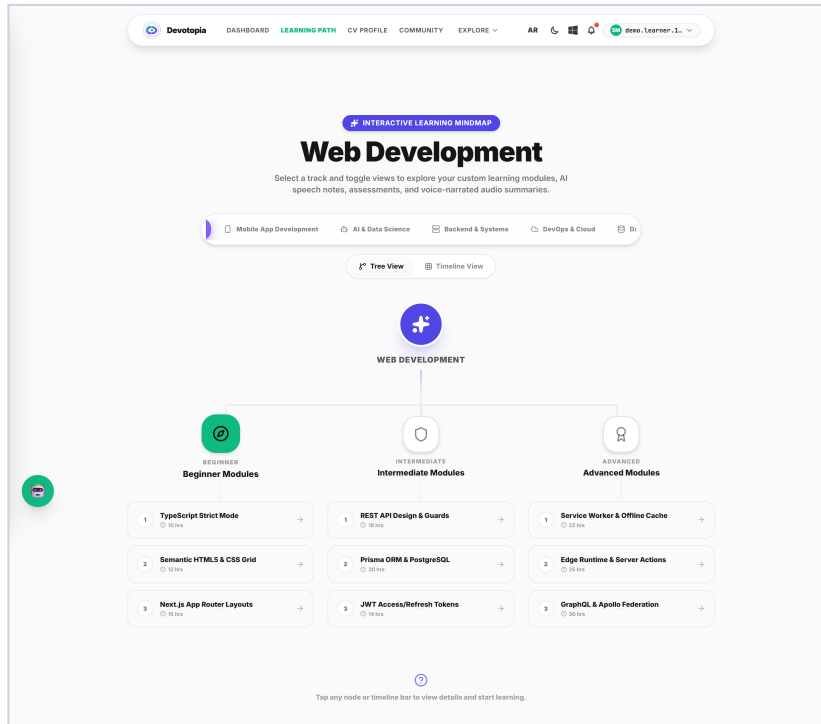
The full dashboard page.

STEP 3 — THE ROADMAP

The curriculum is rendered as a tree. Each node carries a difficulty tier, an hour estimate and a status — locked, in progress, completed or failed. The x/y coordinates for the canvas are produced by the generator itself, so the shape of the tree is part of the generated data rather than a layout guess made in the browser.



The learning path. Beginner, intermediate and advanced branches with per-node status.



The full roadmap view, including the timeline toggle.

4. The adaptive assessment engine

This is the centre of the project. Five questions per module. The engine watches the last two answers and moves the difficulty of the next question up or down; the final score is then weighted by the difficulty the learner actually faced, so five easy questions and five hard ones do not produce the same mark.

The screenshot shows the Devotopia quiz interface. At the top is a navigation bar with the Devotopia logo and links for DASHBOARD, LEARNING PATH, CV PROFILE, COMMUNITY, and EXPLORE. On the right of the bar are icons for AR, a moon, a window, a bell, and a user profile labeled 'quiz.demo.1786...'. Below the navigation bar is a link to '← BACK TO ROADMAP'. The main heading is 'Introduction to Full-Stack Web Developer Foundations'. The quiz area is divided into two columns. The left column contains 'Question 1 of 5' with a 'MEDIUM' difficulty badge. The question text is 'What is a primary concept of "Introduction to Full-Stack Web Developer Foundations" at a medium level?'. There are four options: Option A: Optimizing runtime execution of Introduction to Full-Stack Web Developer Foundations; Option B: Structuring state declarations inside Introduction to Full-Stack Web Developer Foundations; Option C: Implementing standardized interfaces for Introduction to Full-Stack Web Developer Foundations; and Option D: None of the above. The right column features a circular timer showing '29s' and a 'QUIZ QUESTIONS' list with five items: 'Question 1' (highlighted), 'Question 2', 'Question 3', 'Question 4', and 'Question 5'. A small green circular icon with a robot face is in the bottom left corner.

Question 1 of 5. The difficulty badge (MEDIUM) and the per-question timer are visible.

How the difficulty moves

```
// apps/api/src/modules/assessment/assessment.service.ts
// Compute consecutive stats to adjust difficulty for NEXT question
if (answersHistory.length >= 2) {
  const lastTwo = answersHistory.slice(-2);
  const allCorrect = lastTwo.every((ans) => ans.correct);
  const allIncorrect = lastTwo.every((ans) => !ans.correct);
  const currentDifficulty = lastTwo[1]?.difficulty || 'medium';

  if (allCorrect) {
    if (currentDifficulty === 'easy') nextDifficulty = 'medium';
    else if (currentDifficulty === 'medium') nextDifficulty = 'hard';
    else nextDifficulty = 'hard';
  } else if (allIncorrect) {
    if (currentDifficulty === 'hard') nextDifficulty = 'medium';
    else if (currentDifficulty === 'medium') nextDifficulty = 'easy';
    else nextDifficulty = 'easy';
  } else {
    nextDifficulty = currentDifficulty;
  }
}
```

Two in a row moves the ladder one step. A mixed pair holds position.

How the score is weighted

```
// Calculate weighted score: easy=1, medium=1.5, hard=2
session.answers.forEach((ans) => {
  const weight = ans.difficulty === 'easy' ? 1 : ans.difficulty === 'medium' ? 1.5 : 2;
  totalWeight += weight;
  if (ans.correct) earnedWeight += weight;
});

const finalScorePercent = totalWeight > 0
  ? Math.round((earnedWeight / totalWeight) * 100)
  : 0;
const passed = finalScorePercent >= 70;
```

A hard question is worth twice an easy one, so climbing the ladder is what raises the mark.

What happens on pass, and on fail

The branch after scoring is where the product earns its name. Passing unlocks the dependent modules and checks whether the whole track is now finished. Failing does not simply record a failure — it feeds the missed topics back into curriculum generation.

```
// Adaptive outcome: passing unlocks what comes next, failing injects a
// shorter remedial module instead of leaving the learner stuck.
if (passed) {
  await this.unlockNextRoadmapModules(session.userId.toString(), session.moduleId);
  const issuedCert = await this.certService.checkAndIssueCertification(
    session.userId.toString(),
  );
  ...
}

// remedial-node-queue.service.ts
export const REMEDIAL_THRESHOLD = 30; // 30% fail percentage threshold

// assessment.service.ts - recordTopicResult()
result.failPercentage = Math.round((result.failedAttempts / result.attempts) * 100);
if (result.failPercentage >= REMEDIAL_THRESHOLD && !passed) {
  result.status = 'remedial_inserted';
  await this.remedialQueueService.enqueueJob({ userId, topicId, ... });
}
```

The remedial trigger is a threshold on a running fail percentage per topic, not a single bad day.

Both rules are covered by unit tests in `remedial-trigger.spec.ts`, which is possible precisely because the decision is deterministic rather than delegated to a model.

 Devotopia

[DASHBOARD](#) [LEARNING PATH](#) [CV PROFILE](#) [COMMUNITY](#) [EXPLORE](#) 

[AR](#)     quiz.demo.1786... 

[← BACK TO ROADMAP](#)

Introduction to Full-Stack Web Developer Foundations

Question 3 of 5

HARD

What is a primary concept of "Introduction to Full-Stack Web Developer Foundations" at a medium level?

Option A: Optimizing runtime execution of Introduction to Full-Stack Web Developer Foundations

Option B: Structuring state declarations inside Introduction to Full-Stack Web Developer Foundations

Option C: Implementing standardized interfaces for Introduction to Full-Stack Web Developer Foundations

Option D: None of the above

Correct Answer

Simulated explanation for Introduction to Full-Stack Web Developer Foundations (medium).

Next Question →

27s

QUIZ QUESTIONS

 Question 1

 Question 2

 Question 3

 Question 4

 Question 5

Mid-exam. The sidebar tracks position through the five questions.

Devotopia — Verified Skill Roadmaps

15

5. Turning passes into proof

Passed exams accumulate into a passport: a shareable public record of verified milestones with the score and the date each was verified. This is the artefact a recruiter reads, and the same records drive the candidate ranking on the company side.

The screenshot displays the Devotopia 'Verified Skill Passport' for a candidate named Sarah Mitchell. The interface includes a top navigation bar with links to Dashboard, Learning Path, CV Profile, Community, and Explore. The passport itself features a header with the candidate's name, ID, and a 'Share Public Profile' button. Below this, a 'Verified Assessment Milestones' section lists four skills with their respective scores and verification dates: React Framework Architecture (92%), TypeScript Strict Mode Interfaces (95%), Tailwind Design System Tokens (89%), and Next.js App Router Prefetching (87%). A 'Verified Application Code Audits' section at the bottom shows two audits: 'RAG-backed syllabus learning search' and 'NestJS Microservices Gateway', both marked as 'Audit Passed (98+)'. The candidate's overall 'Career Score' is 82% and 'Hiring Readiness' is 94%.

Category	Item	Score	Status	Verified Date
Verified Assessment Milestones	React Framework Architecture	92%	PASS ✓	2026-06-12
	TypeScript Strict Mode Interfaces	95%	PASS ✓	2026-06-15
	Tailwind Design System Tokens	89%	PASS ✓	2026-06-10
	Next.js App Router Prefetching	87%	PASS ✓	2026-06-18
Verified Application Code Audits	RAG-backed syllabus learning search	98+	Audit Passed	
	NestJS Microservices Gateway	98+	Audit Passed	

The verified skill passport, with career score and hiring readiness.

CV BUILDER

A résumé PDF is parsed into a structured profile — personal details, experience, education, projects, skills — and experience bullets can be rewritten by the model for impact. The parse and the enhancement are separate calls, so a failed enhancement never costs the learner their parsed data.

The screenshot displays the Devotopia CV Builder interface. At the top, a navigation bar includes the Devotopia logo, links for Dashboard, Learning Path, CV Profile (active), Community, and Explore, along with user settings for 'demo.learner.1...'. Below this, a sub-header shows 'RESUME / CREATE RESUME' and a search bar. On the right, buttons for 'Upload resume...', 'Cancel', and 'Save' are visible. The main content area is divided into three sections. The left sidebar, under 'JobsSpark', lists 'Dashboard', 'Resumes' (highlighted), 'Job offer', 'Applied job', 'Saved job', 'Message', and 'Notification'. The central form is titled 'Fill in' and includes tabs for 'Guidance', 'Analysis', and 'Matching'. It contains a 'Resume complication' progress bar at 0%, a 'BASIC INFORMATION' section with fields for 'FIRST NAME' (Harry), 'LAST NAME' (Wells), and 'PROFESSIONAL TITLE' (Senior Frontend Developer), and a 'CAREER OBJECTIVES' section. The right section, 'LIVE TYPESET PREVIEW (A4 FORMAT)', shows a preview of the resume with the name 'HARRY WELLS', title 'SENIOR FRONTEND DEVELOPER', and contact information. An 'Export PDF' button is located in the top right of the preview area.

The CV editor. Structured sections rather than a free-text box.

JOBS AND SKILL GAPS

Job matching compares verified skills against a posting's requirements. Where a requirement is missing, it can be injected straight back into the roadmap as new modules — the loop closes back to section 3.

Devotopia

DASHBOARD

LEARNING PATH

CV PROFILE

COMMUNITY

EXPLORE

AR

demo.Learner.1...

VECTOR MATCH RANKING

Hiring Match Pipeline

Jobs recommended based on semantic matching with your Skill Passport credentials.

SEARCH FILTERS

Reset

JOB TITLE / KEYWORD

Search jobs...

☐ Remote Only

MIN MATCH THRESHOLD

88%

Vetted Candidate Profile

Hiring managers see your verified testing badges automatically upon submission. This cuts standard CV vetting time down entirely.

AVAILABLE MATCHES (0)

No matching opportunities found. Try adjusting filters.

AI MATCH ANALYSIS

Frontend Engineer (React & TypeScript)

Lattice HR • Remote

WHY YOU MATCH

Exceptional match! Your verified React (92%) and TypeScript (95%) scores exceed the candidate baseline. Closing CI/CD gaps will achieve 100% compatibility.

REQUIRED SKILLS & GAPS

HTML/CSS

JavaScript

React

TypeScript

Git

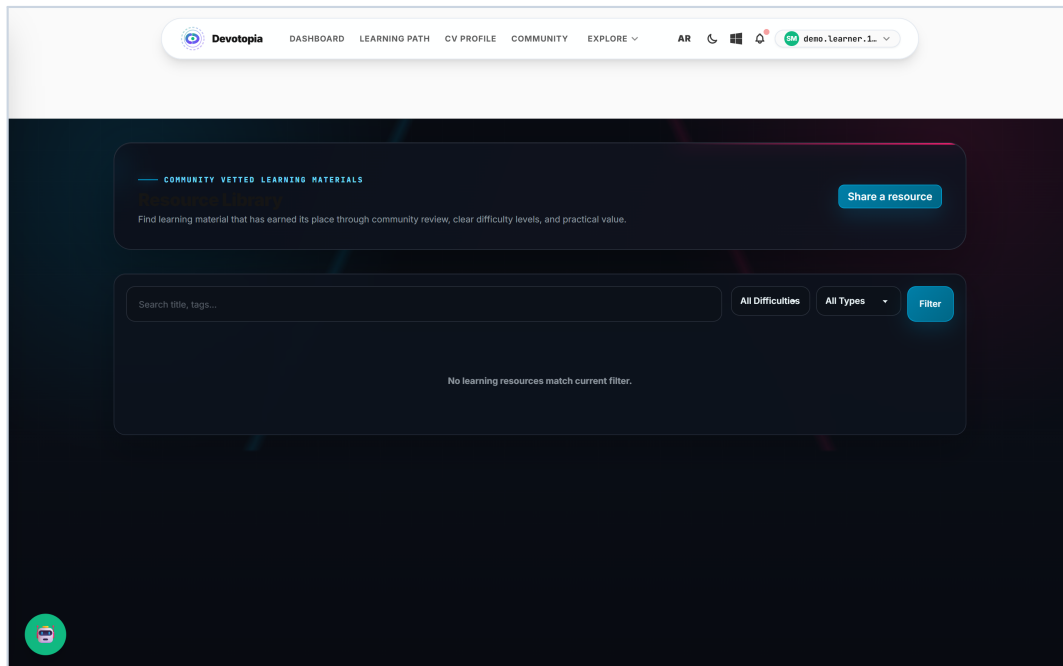
Inject missing skills into Roadmap

Apply with Skill Passport

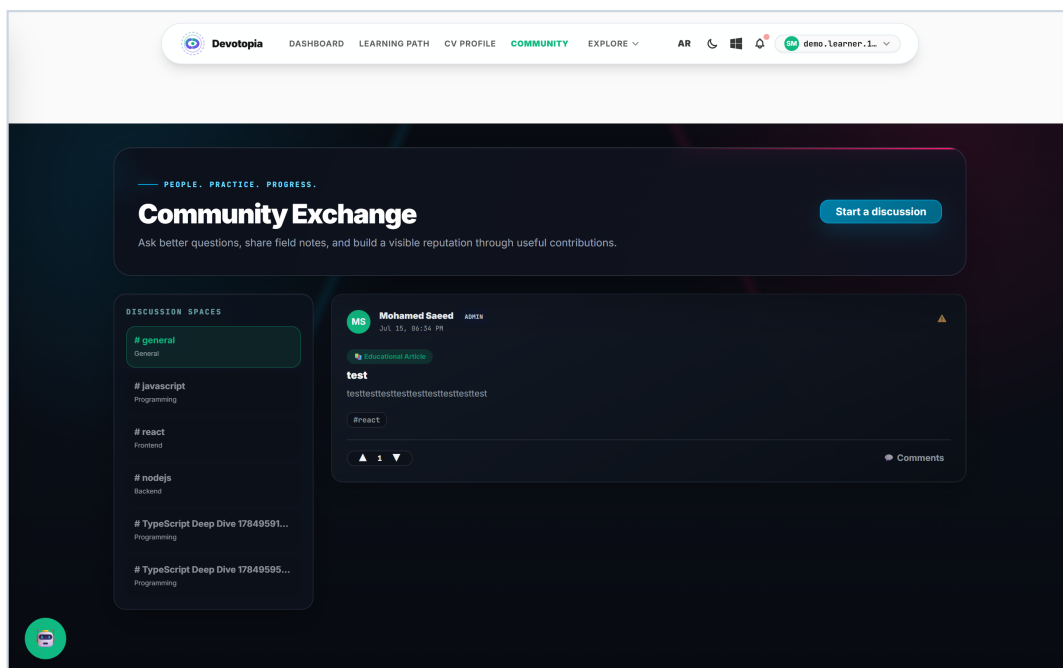
Job matching with the gap comparison.

6. The supporting surfaces

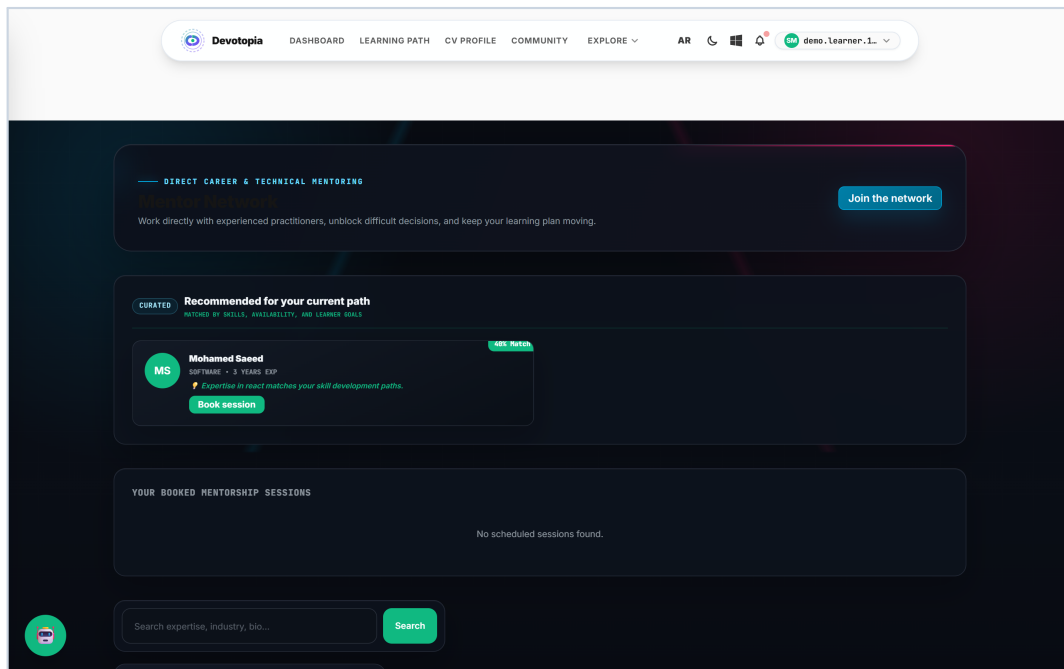
Beyond the core loop the platform carries the surfaces a learner actually needs day to day. Each is a full feature module on the backend, not a placeholder page.



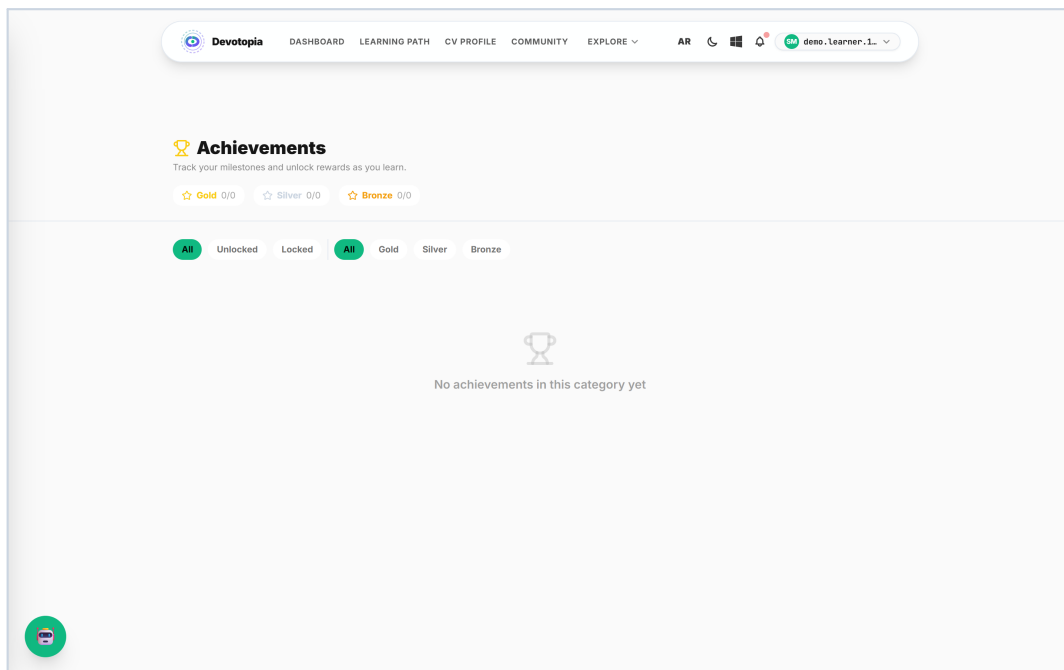
Learning resources, retrieved by semantic search over embedded content.



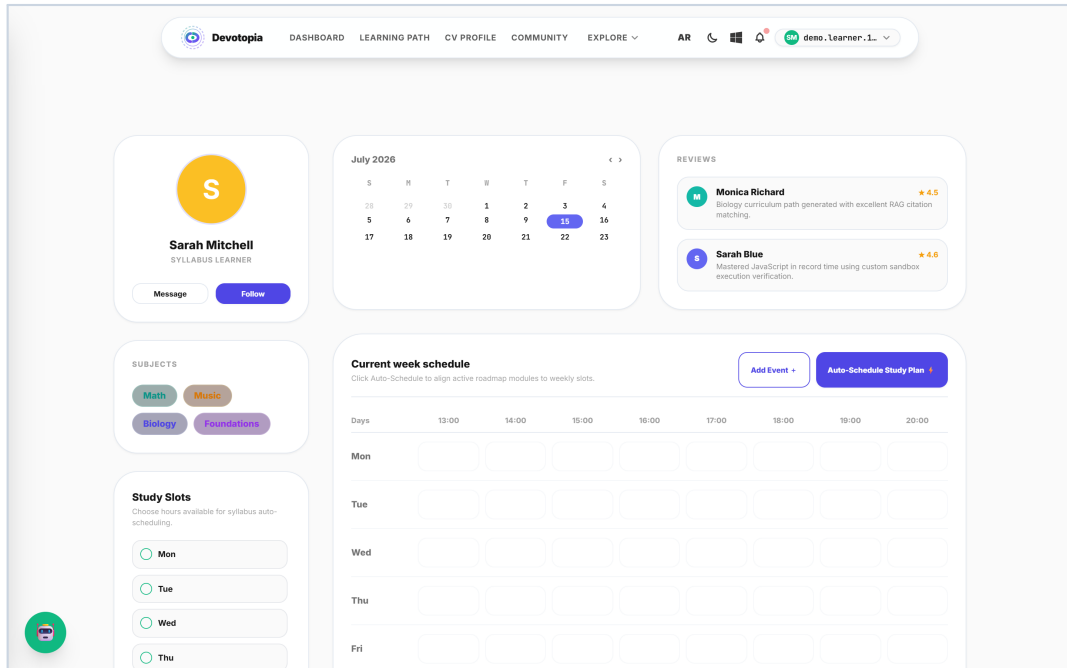
Community discussion spaces.



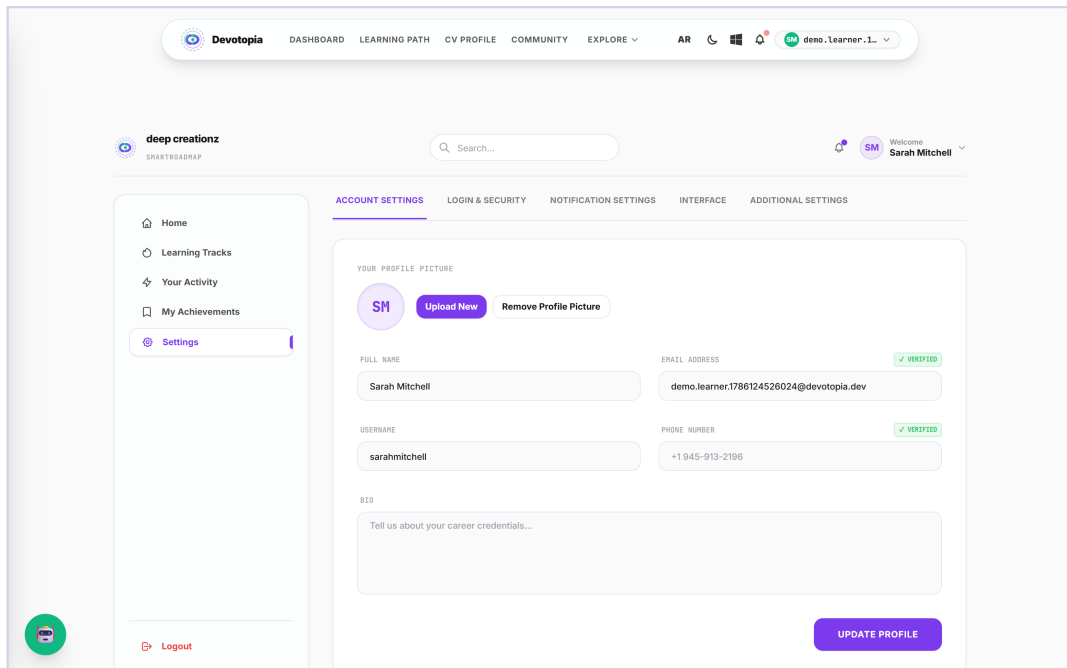
Mentor directory with ratings and session booking.



Achievements — 26 definitions seeded on boot, awarded by event listeners.



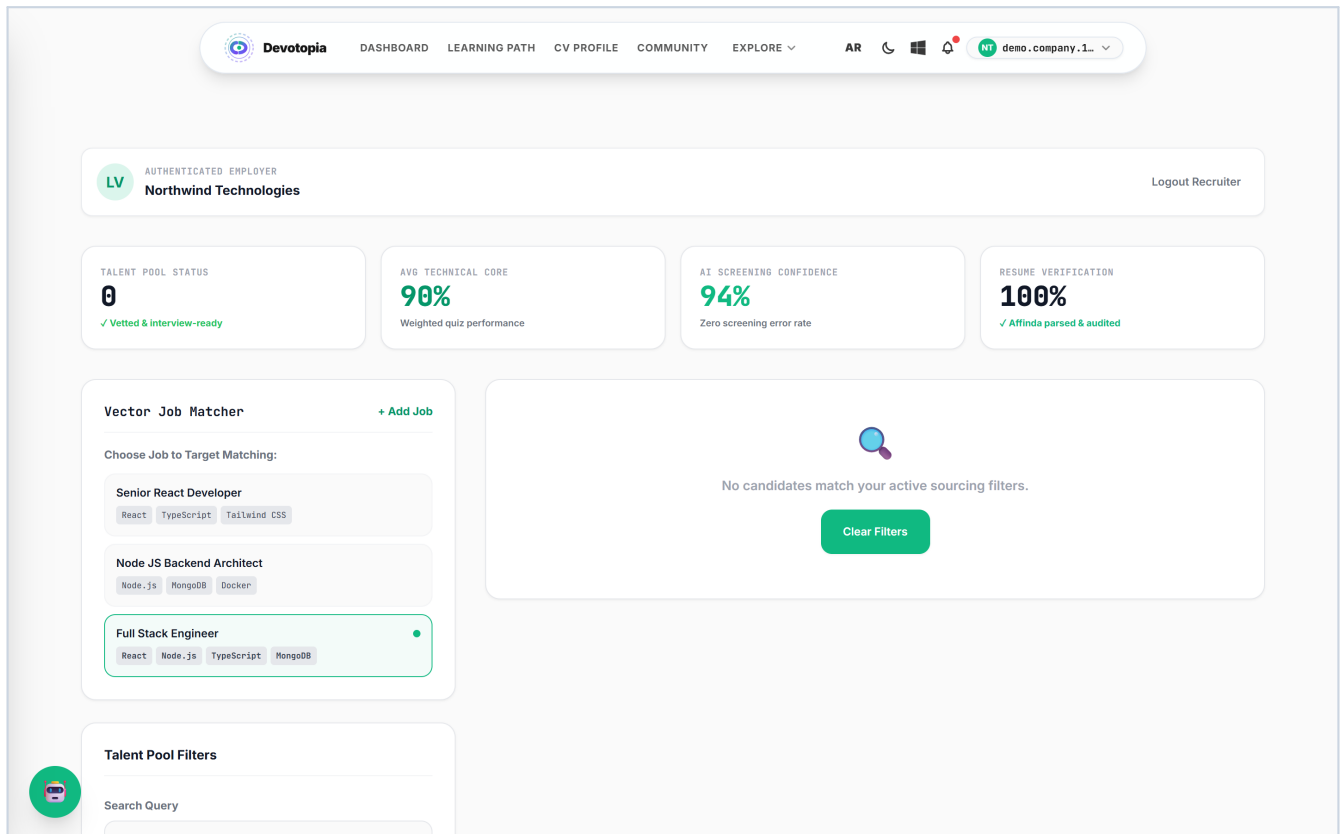
Study calendar and scheduled sessions.



Profile and account settings.

7. The recruiter's side of the platform

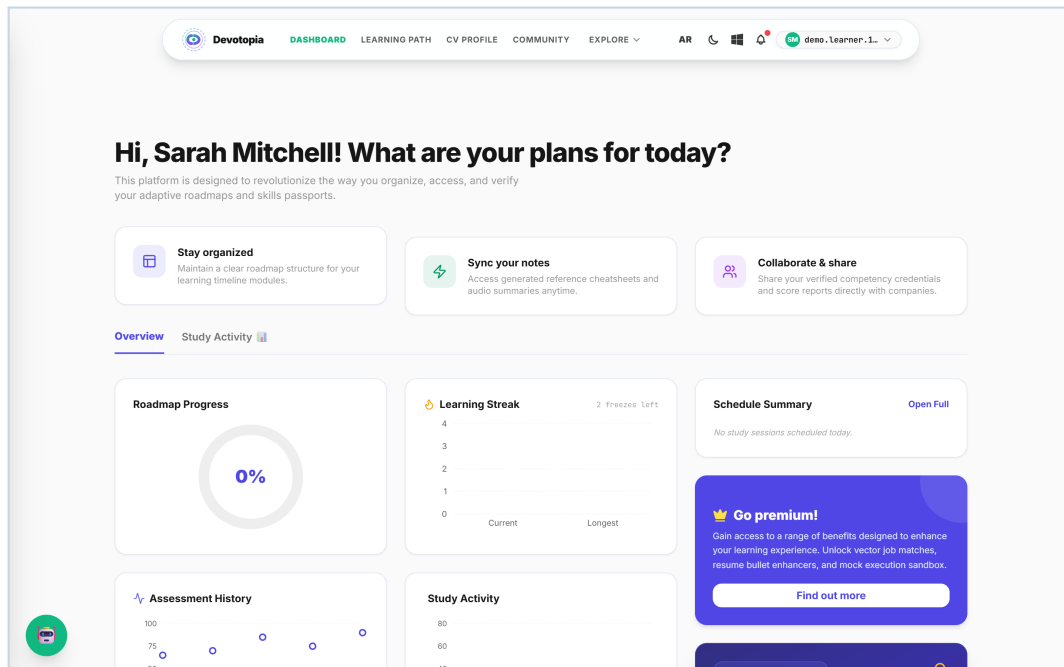
A company account sees a different application from the same deployment. Access is not decided in the browser: the role is re-checked on the server for every request, so navigating to a learner route as a company account returns a 403 rather than a rendered page. During this walkthrough that behaviour was observed directly in the API log — a GET on an admin-only route returned 403 for a non-admin session.



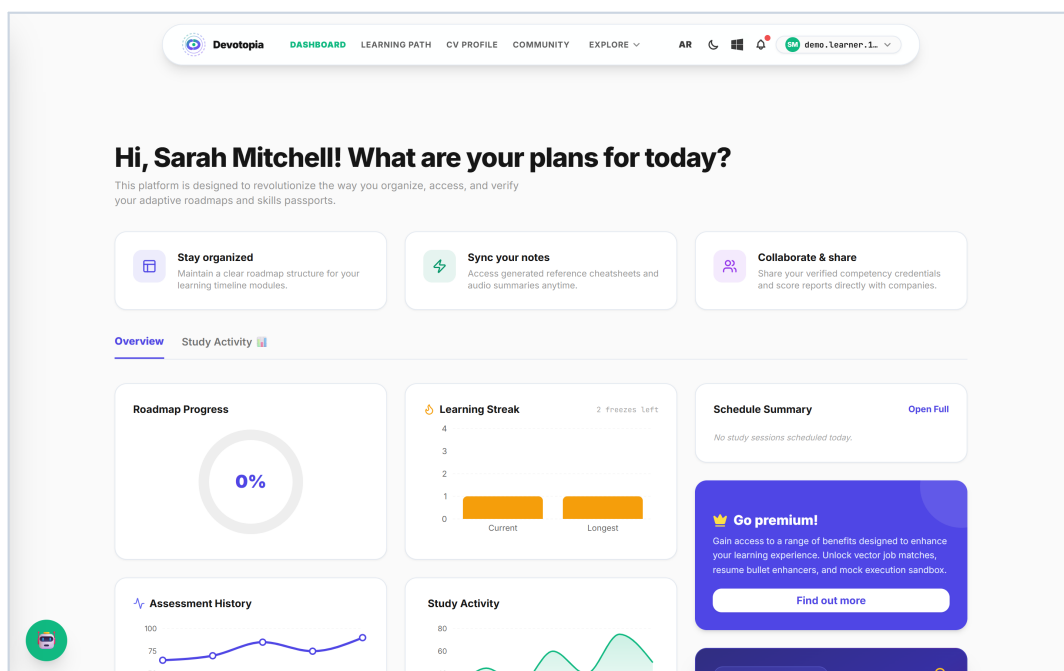
The company dashboard.

8. Two languages, two themes, one layout

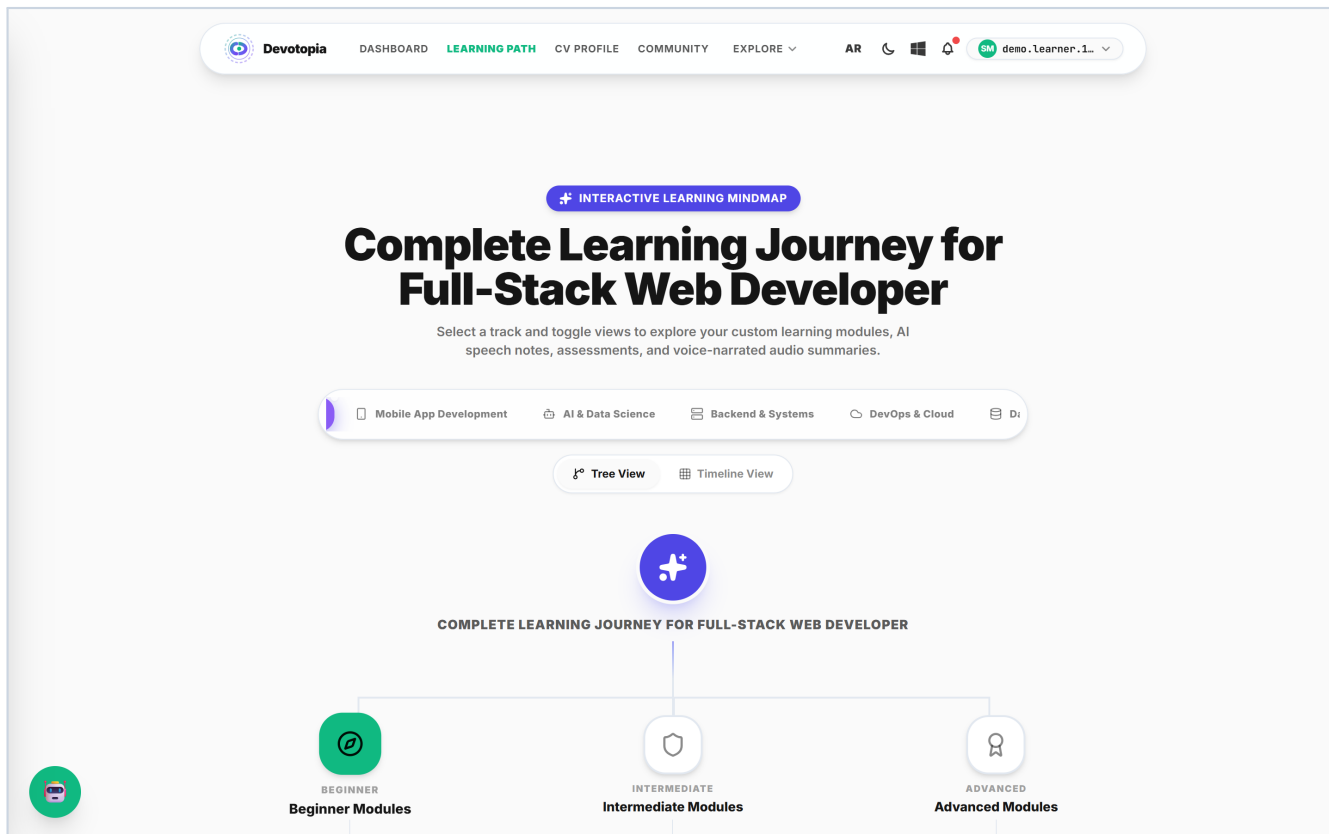
Arabic is not a translation layer bolted on top. Switching locale flips the document direction and the layout follows, because spacing uses logical properties rather than fixed left and right. The same page is shown below in dark mode and in Arabic.



The dashboard in dark mode.



The same dashboard in Arabic, right-to-left.



The roadmap in Arabic — the tree mirrors with the text direction.

9. The AI layer, and what happens when it fails

Every model call goes through one chain. Providers are tried in order and the chain always ends in a deterministic mock, so the product degrades instead of breaking.

```
// apps/api/src/ai/ai-provider.factory.ts
getProvidersChain(): AiProvider[] {
  const chain: AiProvider[] = [];
  const order = ['openai', 'gemini', 'groq', 'huggingface'];
  for (const name of order) {
    const p = this.providers.get(name);
    if (p && p.constructor.name !== 'MockProvider') chain.push(p);
  }
  const mock = this.providers.get('mock');
  if (mock) chain.push(mock); // last resort, never throws
  return chain;
}

// llm.service.ts — a reply is shape-checked before it is accepted
if (Array.isArray(response.modules) && response.modules.length > 0) {
  return response;
}
// otherwise: log at debug level and fall through to the next provider
```

A malformed reply is treated the same as an outage — try the next provider.

This was not theoretical during capture

While these screenshots were being taken, all three configured live providers rejected every request: OpenAI returned 429 with no credits remaining, and both Gemini and Groq returned invalid-credential errors. The chain fell through to the mock provider and the application kept working — roadmaps generated, exams started, scoring ran. That is the design behaving exactly as intended, and it is why the module titles in the screenshots read as generic placeholders rather than as tailored curriculum text.

The practical consequence is stated plainly: the screenshots demonstrate the full product flow, but the curriculum and question text they contain came from the mock provider, not from a live model. Restoring a valid key on any one of the three providers replaces that text with real generated content, with no code change.

10. Running the project locally

```
# from the repository root
npm install
npm run dev          # turbo runs both apps

# API  → http://localhost:3002
# Web  → http://localhost:3001

npm run smoke        # end-to-end + security suite against a live API
```

The two dev servers used for every screenshot in this document.

What had to be fixed to get it running

A clean checkout did not start. Two independent problems were found and resolved, and both are worth recording because they will affect anyone else cloning the repository.

- Backend dependencies were declared but not installed: `@nestjs/event-emitter`, `bullmq`, `node-appwrite` and `web-push`. TypeScript reported 11 module-resolution errors and the API never reached bootstrap.
- Frontend dependencies were missing in the same way: `framer-motion`, `recharts`, `lucide-react`, `@monaco-editor/react` and `lineicons`. Every authenticated page failed to compile, which is why an earlier capture attempt produced only error screens.
- Two services read configuration with `getOrThrow` and abort startup when it is absent — `AppwriteService` needs `APPWRITE_ENDPOINT` and `VoiceAgentService` needs `ASSEMBLYAI_API_KEY`. Placeholder values were supplied at launch rather than written into `.env`.

After those, both servers start clean: the API compiles with zero TypeScript errors, maps 118 routes, seeds 26 achievement definitions and indexes its static search resources on boot.

Honest status of the environment

Works fully	Auth, roadmap generation, adaptive exams, scoring, unlocking, CV, passport, jobs, community, company pipeline, bilingual UI, themes
Degraded	AI text quality — all three live provider keys are currently rejected, so generated content comes from the mock provider
Not running locally	Redis and Qdrant were not started, so the remedial queue used its in-memory fallback and RAG ran in mock mode

11. What to look at first

If someone reviewing this project has ten minutes, these are the files that carry the ideas rather than the plumbing.

assessment.service.ts	The adaptive ladder, the weighted score, the pass/fail branch and the remedial trigger — the core of the product
ai-provider.factory.ts	The fallback chain, and the decision to always end in a mock
llm.service.ts	How a model reply is shape-checked before it is trusted
jwt-auth.guard.ts	Deny-by-default authentication applied globally
auth.dto.ts	Privilege escalation closed at the validation layer
payment.dto.ts	Server-side price resolution
scripts/smoke-test.mjs	The end-to-end security suite: IDOR, spoofing, rate limits

The through-line in all of them is the same choice: let the model generate content, and keep every decision that determines a learner's outcome in deterministic, testable code.